An Empirical Study of API Usability

Marco Piccioni, Carlo A. Furia, and Bertrand Meyer
ETH Zurich, Switzerland
Email: {marco.piccioni, carlo.furia, bertrand.meyer}@inf.ethz.ch

***Abstract*—Modern software development extensively involves reusing library components accessed through their Application Programming Interfaces (APIs). Usability is therefore a fundamental goal of API design, but rigorous empirical studies of API usability are still relatively uncommon. In this paper, we present the design of an API usability study which combines interview questions based on the cognitive dimensions framework, with systematic observations of programmer behavior while solving programming tasks based on "tokens". We also discuss the implementation of the study to assess the usability of a persistence library API (offering functionalities such as storing objects into relational databases). The study involved 25 programmers (including students, researchers, and professionals), and provided additional evidence to some critical features evidenced by related studies, such as the difficulty of finding good names for API features and of discovering relations between API types. It also discovered new issues relevant to API design, such as the impact of flexibility, and confirmed the crucial importance of accurate documentation for usability.**

I. Introduction

It is not by chance that software has become so complex, but by building over the solid underpinnings of abstraction and modularity. Modern software is a complex composition of library components, whose elementary functionalities are combined to achieve domain-specific applicative goals. A component's *interface* consists of features that clients can call; the combination of interface features and a description of their usage protocol and semantics is what is normally called API: Application Programming Interface. Using library components solely based on their APIs makes it possible to abstract away implementation details and practice modularity; therefore, learning and using APIs are everyday tasks for software developers. From a research perspective, it is interesting to understand what makes APIs easy or hard to use; this is the overall goal of this paper.

APIs are a fundamental interface for the interactions between programmers and computers [1]. Thus, API usability impacts software development quality [2], [3]: usable APIs are more intuitive, require less documentation browsing, and encourage reuse, thus increasing developers' productivity. Conversely, APIs that are hard to use reduce programmer productivity and quality of the final product, as shown, for example, by measuring requests for technical support [3].

Previous empirical studies of API usability—reviewed in Section VIII—point to some critical issues such as the ease of discovering relationships between types offered by an API and how to instantiate new objects. With our work, we find additional evidence to confirm or dispute these issues, as well as to find out new criticalities not detected in previous work.

To this end, the first contribution of our paper is the design of an empirical study of API usability. The study's research questions—described in Section II—are based on the findings of previous studies with the goal of corroborating and extending them. Section III describes the specific study design, which combines two methodologies useful for assessing usability. The first is the notion of *cognitive dimensions*: elements that characterize the expectations of users and what an API actually provides, such as the abstraction level and the consistency between different API functionalities [4]. In our study, we collected and interpreted the feedback of participants according to the cognitive dimensions. The second methodology used in our study design is based on *usability tokens*, an original classification of the participant reactions (such as "surprise" or "incorrect choice") as they try to understand and use API functionalities to perform the assigned tasks. We tallied usability tokens when replaying the videos of the participants' performances. As we discuss in Section VI, combining cognitive dimensions and usability tokens leads to a rich characterization of API usability issues.

The second paper contribution is an actual execution of the study to evaluate the usability of a persistence library written in Eiffel. The study—which we detail in Section IV—involved 25 participants, including both students and professional developers. The participants solved five tasks requiring to access, modify, and query a relational database through the services offered by the persistence library API; we recorded their performance to be able to analyze it postmortem. After each session, we asked the participants about their experience and expectations using a structured interview whose questions were characterized by the cognitive dimensions.

Section V analyzes the study results based on the analysis of performance and interviews; and Section VI discusses the findings in a more general context, which include:

- Finding descriptive, non-ambiguous names for API features is problematic given that programmers may be used to different terminologies.
- Discovering relations between API types (classes) requires significant effort; simple designs are beneficial, especially to less experienced programmers.
- Accurate and complete documentation is a crucial issue for API usability; all the major usability flaws discovered in our study trace back to unsatisfactory documentation.[1]
- Flexibility is a double-edged sword in API design: experienced programmers can take advantage of it, but it may confuse those with less practice.

Finally, Section VII mentions possible threats to the validity of the study; and Section IX concludes.

[1] As documentation, we mainly consider: public method signatures, comments, and contracts of the class features.

II. Research Questions

We organize the study around four main research questions, which characterize fundamental aspects of usability: understandability, abstraction level, reusability, and learnability. The questions cover the critical aspects emerged in previous API usability studies (see Section VIII) but are sufficiently general to make room for new findings and specific aspects emerged in our study (see Sections V and VI).

- **RQ1** What is the effort required to understand the semantics of API features based on their names and documentation?
- **RQ2** Does the API's abstraction level cater to usability?
- **RQ3** Does the API's design facilitate reuse and conciseness in client code?
- **RQ4** Can API usage be learned easily and incrementally?

RQ1 addresses the fundamental problems tackled by programmers using the services offered by an API: what they have to do to understand what each feature of the API represents and how it should be invoked. This question also encompasses specific issues such as whether the API feature names are descriptive, whether the relationships between API elements are clear and unambiguous, and how to access the features through object creation, method call, or other means.

RQ2 evaluates whether the level of abstraction is good for usability. On the one hand, it should guarantee that programmers can proficiently use the API without knowing (or assuming) implementation details. On the other hand, the abstractions should match the conventions and practices of programmers, without being elegantly abstract at the expense of understandability and other practical concerns. Summarizing with a slogan, this research question asks whether the API "makes simple things simple, and complex things possible" [5].

RQ3 determines to what extent the client code that can be written using the API is concise, terse, maintainable, and extensible. Concretely, this question addresses problems such as: if we have an application using some API features and slightly vary or generalize its requirements, how hard is it to adapt the application to meet the extended requirements?

RQ4 targets the API learning process to see if it can be incremental (that is, gradual and not requiring disproportionate efforts) and whether performing a certain programming task using the API has a positive impact on performing other, related but different, tasks. This question is related to RQ1 and RQ3 but emphasizes the learning process rather than its practical outcomes.

III. Study Design

Usability is a multifaceted feature which is hard to assess reliably because it involves somewhat subjective aspects such as the specific habits and styles of individual programmers. To address such problems, the cognitive dimensions framework [4] suggests to evaluate usability operationally based on comparisons. First, we outline a number of aspects that are sufficient to characterize the multiple facets of usability in our specific domain. Then, we base the assessment on the *comparison*, for each aspect, between the expectations of users and what the system actually provides. For example, regarding incremental learnability in APIs, users may expect to be able to execute incomplete code to obtain feedback on API behavior, whereas the API might require specific complete call sequences before the code is executable without errors; this would signal an imperfection in the API's usability.

Our empirical study follows the guidelines of the cognitive dimensions framework, and more specifically Clarke's dimensions of API usability [6]. It is based on the execution of programming tasks that require the participants to write client code combining API features to achieve certain functionalities; the comparison between expectations and reality is based on the participants' performance on the programming tasks. With the goal of having a multidimensional assessment, which helps reduce the impact of subjective perceptions, we collect data about the discrepancies between user expectations and actual system features from two different sources.

After completing the tasks, we conducted a structured interview with the participants involving a fixed list of questions about their experience and expectations. This explicit feedback provided by programmers through the interview highlights their perceived usability, the difficulties they experienced, and the features that explicitly appreciated. Section III-A describes the interview questions in some detail and discusses how they articulate the research questions of Section II.

Independent of the interview data, we also collected feedback given implicitly by the participants during their performance. To this end, we asked them to follow the thinking-aloud protocol and recorded the video and audio of their performances. The thinking-aloud protocol is based on Ericsson and Simon's seminal work [7] and consists in having the participants declare in speech their mental process as it develops, including the doubts and questions they have, the solution strategies they consider, and the reasons that justify their decisions. This protocol is widely used in usability testing [8], because it makes the external observer aware of the cognitive processes leading to a certain performance. In our study, we perused postmortem the recordings of the performances searching for episodes revealing the expectations of programmers and how they compared to the API features. As we describe in Section III-B, we classify the episodes in *usability tokens*, which express mismatch between user expectations and actual performance. The tokens make the implicit data collection more uniform and aligned to the issues targeted by the research questions.

A. Structured Interview

Following Clarke's guidelines [6], and reflecting the research questions of Section II, the interview consists of twelve questions which characterize various aspects of usability as assessed by users. The questions are as follows, roughly grouped by the research question they target (although some of them target multiple research questions).

Questions regarding RQ1 (understandability):

1) Do you find that the API types map to the domain concepts in the way you expected?
2) Do you feel you had to keep track of information not represented by the API to solve the tasks?

3) Does the code required to solve the tasks match your expectations?

Questions regarding RQ2 (abstraction):

4) Do you find the API abstraction level appropriate to the tasks?
5) Did you need to adapt the API (inheriting from API classes, overriding default behaviors, providing non-API types) to meet your needs?
6) Do you feel you had to understand the underlying implementation to be able to use the API?

Questions regarding RQ3 (reusability):[2]

7) Does the amount of code required for each task seem about right, too much, or too little for you?
8) How easy was it to evaluate your own progress (intermediate results) while solving the tasks?
9) Do you feel you had to choose one way (out of many) to solve a task in the scenario?
10) Do you feel you would have to change much in your code to access another kind of persistence store, or write another query?

Questions regarding RQ4 (learnability):

11) Once you performed the first two tasks, was it easier to perform the remaining tasks?
12) Do you feel you had to learn many classes and dependencies to solve the tasks?

We use the cognitive dimensions framework to formulate questions that may discover the existence of problematic issues with the API, rather than aiming at "proving" its usability. This angle—reminiscent of the way testing cannot not prove programs correct but discover the existence of errors—is aligned to the best practices of using the cognitive dimensions framework [9], [10].

B. Usability Tokens

We classify the salient events occurring during participant performances into five tokens. While the tokens describe different dimensions, the same event may be associated to multiple tokens when it reveals aspects of different nature. For each token, we make simple examples based on a hypothetical data-structure library to make the description independent of the specific application domain used in our study (namely, persistence).

Token "surprise": the developer discovers aspects of the API that clearly go against her expectations and original intuitions. For example, the generic container classes may not be usable with primitive types (e.g., integers) but only with object types; this requires a special treatment for certain types, which the user may find surprising.

Token "choice": the developer is faced with a choice and she must understand the alternatives to proceed in the right way. For example, the element at a given position in a list may be accessible either with a method call *s.at(i)* or using the array notation $s[i]$; the user may have to understand if the two options are equivalent and choose the most appropriate one (in terms of correctness and code readability).

Token "missed": the developer's activity shows that she has missed some important abstraction or feature of the API, which would have been useful to effectively solve the current task. For example, the API may offer a feature to remove the first occurrence of a given element in the list, but the developer may miss it and implement the same functionality by first searching and then deleting.

Token "incorrect": the developer uses the API incorrectly, in a way that introduces errors or implements an incorrect functionality. For example, popping elements from a stack without checking for emptiness may lead to errors when the stack is empty.

Token "unexpected": the developer uses the API in a way undocumented or otherwise unforeseen by the API intended design. Unlike the "surprise" token, this token adopts the point of view of API designers. For example, the user may inherit from a stack class and override the push method to allow insertion in the middle of the stack; this is not necessarily wrong, but violates the original design abstraction.

The usability tokens are largely orthogonal to the cognitive dimensions (on which the interview questions are based), in that the same token may characterize events involving different cognitive dimensions. For example, an element of surprise may reveal inconsistent abstractions, poor understandability, or both. This helps make the data from the interviews largely independent of the usability token classification.

IV. Study Setup

We implemented the study design discussed in Section III to assess the API usability of a recently developed persistence library for Eiffel. In the following subsections, we give a few details about: the library and its API; the programming tasks involving the API which we submitted to the participants; the demographics of the participants; and the protocol used to carry out the experiments.

A. Persistence Library API

We evaluated the API of ABEL (A Better EiffelStore Library), a library offering an object-oriented interface to persistence-related functionalities like storing and retrieving data using files (serialization) and relational databases. The first author developed ABEL as part of his PhD work [11]. ABEL is written in Eiffel and is meant as an improvement over the persistence services offered by Eiffel's standard libraries. As it introduces features normally available in other mature persistence frameworks such as Spring [12] and Hibernate [13], it should be usable by "standard" programmers, and hence it is interesting to assess its usability.

ABEL consists of 81 classes grouped into 11 clusters (roughly equivalent to Java packages) for a total of roughly 10500 lines of code, comments, and assertions. In Eiffel, it is customary to use assertions in the form of contracts (pre- and postconditions, and class invariants) to specify the essential requirements and behavior of methods. Therefore, when browsing the API of ABEL, developers can see the signature,

[2]Question 10 is specific to the domain of the API we evaluated, but it can be easily generalized to different application domains.

comments, and contracts of its features, which constitute the documentation of the API's functionalities.

B. Tasks

The study participants solved five tasks involving accessing a relational database using ABEL's features. Solving the tasks requires to store objects of a simple class *PERSON*, also given to the participants, and includes operations found to be critical in previous work [14], [15]. An outline of the five tasks follows.

Task 1: initialize a *REPOSITORY* instance through a factory method; use the *REPOSITORY* instance through a *CRUD_EXECUTOR* to insert instances of *PERSON* into the repository.

Task 2: instantiate a *QUERY* class; use it through the *CRUD_EXECUTOR* to retrieve *PERSON* objects from the repository; and inspect the results.

Task 3: change the state of the *PERSON* objects; update the repository with the new objects using the *CRUD_EXECUTOR*.

Task 4: delete one of the *PERSON* objects and remove it from the repository using the *CRUD_EXECUTOR*.

Task 5: execute a complex query that requires selection criteria. Developers can choose between using "predefined" criteria (implemented using strings) and using function objects (called "agents" in Eiffel, and similar to C#'s delegates).

C. Participants

We recruited 25 participants including 10 computer science students at ETH (6 bachelor's and 4 master's), 7 researchers pursuing a PhD (2 in our group, 2 in other groups of the computer science department of ETH, 2 in the computer science department of other universities, and 1 from the robotics group in the mechanical engineering department of ETH), 2 post-doctoral researchers also at ETH (1 from the computer science department, and 1 from the mechanical engineering department), and 6 professional programmers working for various software companies mostly in the Zurich area. All the participants to the study were unpaid volunteers; the only requirement on our side was that they had at least one year of experience with object-oriented programming.

The following table shows some statistics about their background in terms of: years of programming experience with object-oriented programming languages; years of experience with the Eiffel language; and years of experience as professional programmers.

	min	*median*	*max*	*stddev*
Object-Oriented	1	5	22	4
Eiffel	0	1	18	5
Professional	0	2	21	5

The statistics show that the participant pool covers a wide range of experiences; all participants, however, have a programming background sufficient to make the experiment meaningful. Furthermore, we ascertained that all the participants had at least some familiarity with the basic operations of a relational database, but none of them had used the ABEL library before.

D. Protocol

Each participant performed in an individual session taking place in an isolated office. The first author, henceforth the "proctor", administered all the sessions according to the following protocol.

The proctor starts with a brief overview of the whole process. He then administers a fifteen-minute tutorial showing the basics of the Eiffel language and of the EiffelStudio IDE running on the laptop used for all the experiments. In particular, the tutorial highlights the IDE functionalities useful to browse library documentation (consisting of classes, method signatures, header comments, and contracts) and to inspect the inheritance relations among classes (such as listing all ancestors, descendants, clients, or suppliers of a given class). The only documentation about ABEL available to the participant during the study is accessed using these IDE features.

After the tutorial, the proctor describes the thinking-aloud protocol (Section III) and asks the participant to stick to it during his or her performance. Then, the proctor opens a project consisting of the *PERSON* class and a "main" client class including a terse description of the five tasks as comments (see Section IV-B). The project is set up with the "full void-safety" flag, which entails that the compiler statically checks for possible dereferencing of void references (null references in Java); this helps avoid basic programming mistakes and lets the participants focus on correctly using the API functionalities. Finally, the proctor starts the audio/video recording of the session and invites the participant to begin.

During the experiment, the proctor sits in the same room avoiding interactions with the participant. In a few cases, some of the participants asked for the proctor's instant help, mainly with using functionalities of the IDE or with the syntax of the Eiffel language. The proctor only answered requests that were independent of the specific tasks or the API functionalities, giving the minimal information necessary to proceed. Section VII discusses this potential threat, to demonstrate that its impact on the soundness of the experiments is negligible.

The proctor lets the experiment continue until the participant completes all the five tasks. There is no time limit because the focus of the experiments is assessing usability, not measuring programming efficiency (something would have made not much sense anyway, given the heterogeneous experience of the participants). We still report the time taken by the participants to show that it always was within a reasonable range: the fastest participant finished in 32 minutes, the slowest in 118 minutes, the median time was 70 minutes, and the standard deviation 22 minutes.

After completion of the tasks, the proctor interviews the participant asking the questions of Section III-A and recording his or her answers. This concludes the experimental session.

V. Results

We present the results of the study in subsections corresponding to the four research questions of Section II. For each question, we discuss the data both from the interviews and from the usability tokens, which is summarized in Tables I and II.